

TestStand Crash Course

Generated by Doxygen 1.16.1

1 Start Here	1
1.1 Introduction	1
1.1.1 About this Guide	1
1.1.2 Why Use TestStand?	1
1.2 Getting Started	1
1.2.1 Open TestStand	1
1.2.2 Familiarize Yourself	2
2 Terminology	3
3 Configuration & Options	5
3.1 Sequence File Properties	5
3.1.1 File Globals	5
3.1.2 Format	5
3.2 User Manager	6
3.3 Station Options	6
3.3.1 Execution	6
3.3.2 Model	7
3.4 Search Directories	7
3.5 Model Options	8
4 Developing a Test Sequence	9
4.1 Variables	9
4.1.1 Variable Types	9
4.1.2 Variable Scopes	10
4.1.3 Working with Variables	10
4.1.3.1 Variables	10
4.1.3.2 Variables into Code Modules	11
4.2 Test Steps	11
4.2.1 Test Step Types	11
4.2.1.1 Action	11
4.2.1.2 Pass/Fail	12
4.2.1.3 Sequence Call	12
4.2.1.4 Statement	12
4.2.1.5 Flow Control	12
4.2.1.6 Database	13
4.2.2 Working with Test Steps	13
4.2.2.1 Creating a Test Step	13
4.2.2.2 Editing a Test Step	13
4.3 Types	14
4.3.1 The Types Pane	14
4.3.2 Working with Types	14
4.3.2.1 Creating a Type	14

4.3.2.2 Data Type Properties	15
4.4 TestStand Flows	15
4.4.1 Sequence Flow	15
4.4.2 Test Step Flow	16
5 Running a Test Sequence	19
5.1 Program Execution	19
5.1.1 Step Run Modes	19
5.1.2 Running the Sequence	19
5.1.3 Running Selected Steps	19
5.2 Debugging	19
5.2.1 Sequence Analyzer	20
5.2.2 Breakpoints & Watch Expressions	20
5.2.3 Step Commands	20
6 Tips & Tricks	21
7 TestStand API	23
8 Arxtron TestExec	25
9 Change Log	27
Index	29

Chapter 1

Start Here

1.1 Introduction

1.1.1 About this Guide

Welcome to the Arxtron TestStand crash course. The aim of this guide is to succinctly summarize the key functionalities of TestStand as used at Arxtron; however, it is not intended to be exhaustive. For further support, you can ask a colleague, or consult the online [TestStand Manual](#).

If this is your first time reading the guide, it is recommended that you follow it sequentially and follow along in TestStand when applicable.

1.1.2 Why Use TestStand?

TestStand provides a structured and scalable framework for developing, executing, and maintaining automated test sequences. It enables standardization across projects, improves code reusability, and simplifies debugging and reporting of test results.

1.2 Getting Started

1.2.1 Open TestStand

1. Press the Windows key and search for "TestStand Version Selector". Open the application.
2. Under "Installed TestStand versions", there will be a dropdown listing the currently installed versions. If TestStand 2017 is not yet installed, talk to IT. Otherwise, select 2017.
3. Select "Make Active", and then select "Launch 32-bit".
4. When TestStand launches, it will prompt you for credentials. By default, the username should be "administrator" and the password should be left blank.

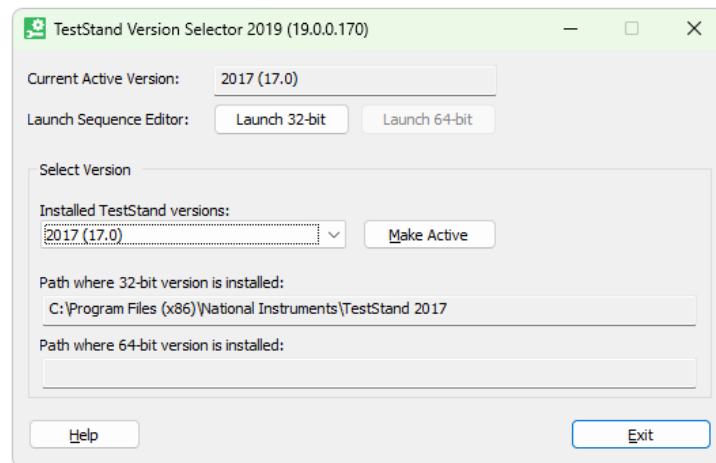


Figure 1.1 TestStand Version Selector

1.2.2 Familiarize Yourself

Once TestStand is open, select **View > Reset UI Configuration**. This will ensure that your UI layout is the same as described in this guide. However, the TestStand UI is completely customizable and can be adjusted to your liking.

Take a few moments to familiarize yourself with the general layout of the program.

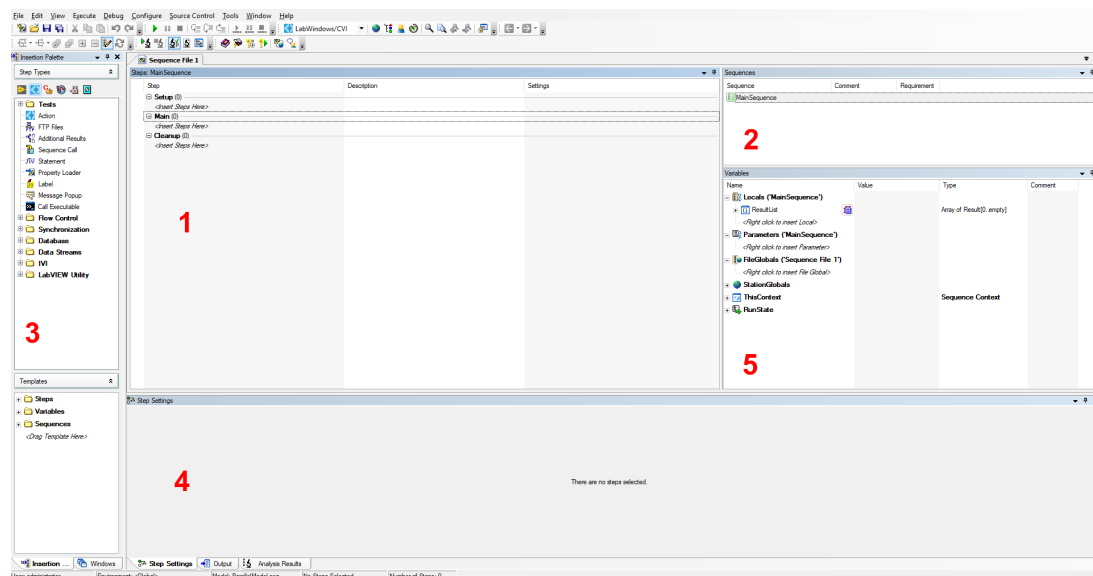


Figure 1.2 Default TestStand Layout

The interface is divided into the following panes:

1. **Steps Pane:** this is where test steps are sequenced.
2. **Sequences Pane:** contains a list of all sequences within the sequence file.
3. **Insertion Palette:** contains steps that can be dragged into the steps pane.
4. **Step Settings Pane:** when a test step is selected, this pane can be used to view and edit step settings.
5. **Variables Pane:** all variables are listed here.

Chapter 2

Terminology

Briefly familiarize yourself with the following terms before continuing, as they will be referenced throughout the guide.

Sequence File

A file type that can be opened via a `test executive`. For example, TestStand works with .seq files. A sequence file typically contains one or more `test sequences`.

Test Executive

A high-level application that can be used to manage, write, and execute `test sequences`. TestStand is an example of a `test executive`.

Test Sequence

A collection of `test steps`. Often simply referred to as a "sequence" within TestStand; be sure to distinguish between "sequences" and `sequence files`.

Test Step

A singular test, often taking place within a `test sequence`.

UUT

Stands for Unit Under Test.

Chapter 3

Configuration & Options

TestStand contains a number of menus containing configuration settings and options. This section of the guide will briefly list the most important settings and their recommended states.

3.1 Sequence File Properties

Can be accessed via **Edit > Sequence File Properties**.

3.1.1 File Globals

The dropdown allows the user to select whether variables within the FileGlobals scope should be shared between executions (especially important when running in **Parallel Mode** or **Batch Mode**). Adjust this setting on a case-by-case basis.

3.1.2 Format

The dropdown allows the user to choose between saving the file in INI, XML, or binary format. For now, it doesn't matter.

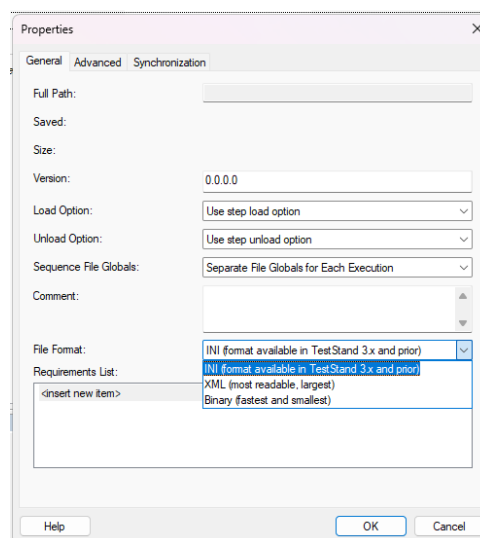


Figure 3.1 Sequence File Properties Menu

3.2 User Manager

Can be accessed via **View > User Manager** or **Ctrl+U**. The user manager is used to add and remove users and user groups, as well as edit information such as passwords and privileges. If you wish, add a new user profile for yourself, making sure to grant yourself admin privileges.

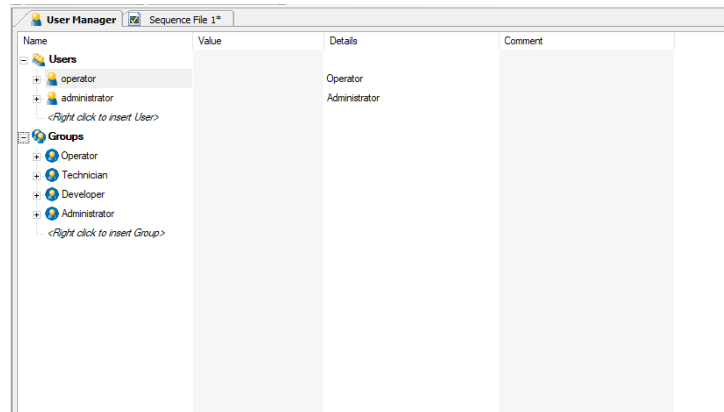


Figure 3.2 User Manager

3.3 Station Options

Can be accessed via **Configure > Station Options**.

3.3.1 Execution

This panel contains important settings related to test sequence execution. Enable Breakpoints should be enabled, as well as Allow Break While in Code Modules. This ensures that test sequences can be properly debugged, especially when stepping into code modules. Additionally, Enable Tracing should be turned on.

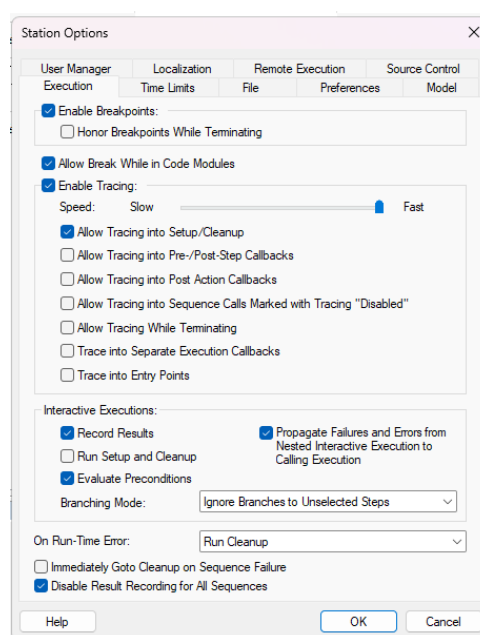


Figure 3.3 Station Options Execution Panel

3.3.2 Model

This panel contains settings related to the station model. TestStand supports three station models:

1. **Parallel:** Runs the test sequence on multiple UUTs independently and concurrently, testing a new UUT whenever a test socket becomes available.
2. **Batch:** Runs the test sequence on multiple UUTs similar to parallel, but synchronizes execution such that tests start at the same point and wait until they reach specific synchronization points
3. **Sequential:** Runs the sequence on one UUT at a time.

At Arxtron, we typically use the Parallel station model, but it's good to be familiar with all three models.

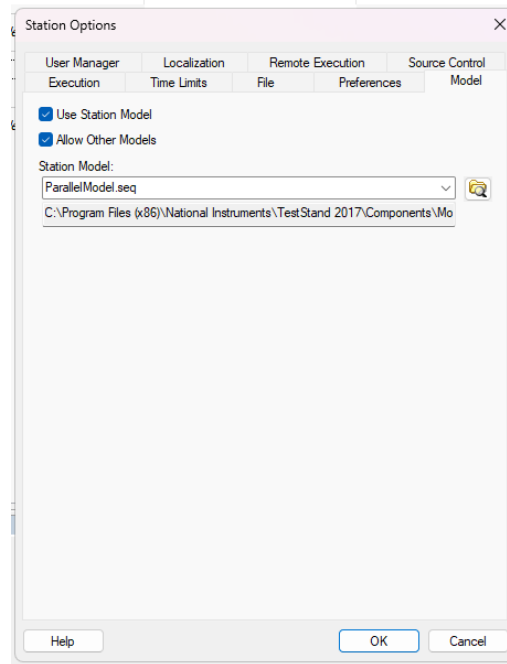


Figure 3.4 Station Options Model Panel

3.4 Search Directories

Can be accessed via **Configure > Search Directories**. Used to configure TestStand's search directories. Ensure that ****C:\Arxtron\Libraries**** is added, with the "Subdirs" option selected; this allows TestStand to find and reference Arxtron library .dlls.

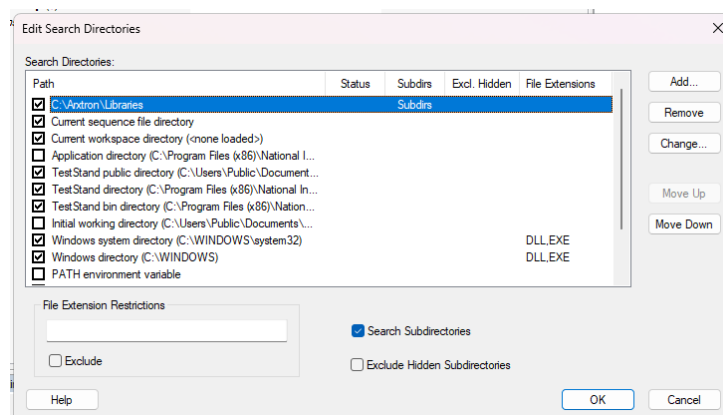


Figure 3.5 Search Directories Menu

3.5 Model Options

Can be accessed via **Configure > Model Options**. Primarily used to select the number of "Test Sockets", or the max number of UUTs that can be tested simultaneously.

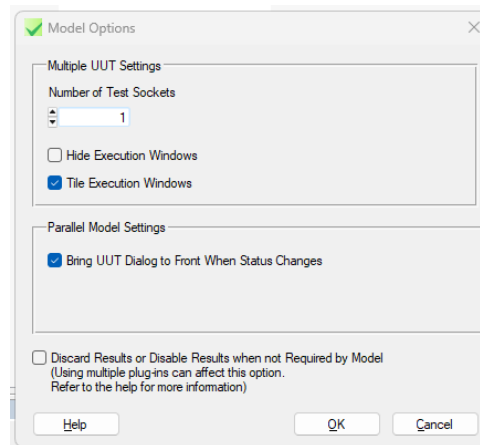


Figure 3.6 Model Options Menu

Chapter 4

Developing a Test Sequence

- [Variables](#)
- [Test Steps](#)
- [Types](#)
- [TestStand Flows](#)

4.1 Variables

Variables in TestStand work similarly to variables in most programming languages. The following section will detail how they work.

4.1.1 Variable Types

- **Number:** encompasses all numeric types.
- **String:** note that there is no "char" variable type, only string.
- **Boolean**
- **Object Reference:** references an object (similar to a pointer).
- **Container:** a structure that can hold members of various data types (similar to a C struct).
- **Type:** a user-defined datatype.
- **Array**

4.1.2 Variable Scopes

From smallest to largest:

- **Step**: Only exists within the current test step.
- **Local**: Only exists within the current sequence.
- **Parameters**: Must be passed into the current sequence by the caller sequence.
- **FileGlobals**: Global variables that exist everywhere within a sequence file.
- **StationGlobals**: Variables shared across the "station" (all sequence files have access).
- **ThisContext**: The current sequence "context".
- **RunState**: A broad grouping encompassing the TestStand engine and Context, as well as other items.

Note: **ThisContext** and **RunState** are typically only used when working with the API. However, feel free to familiarize yourself with them.

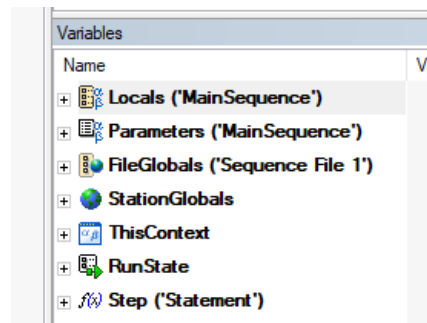
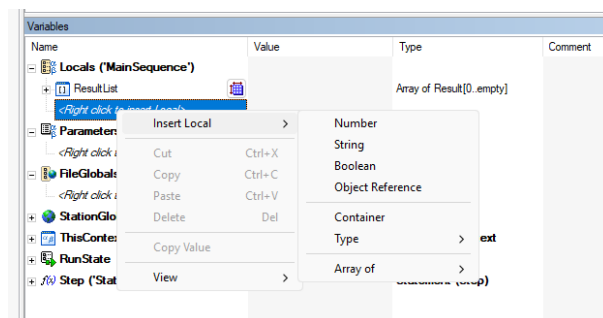


Figure 4.1 TestStand Variable Scopes

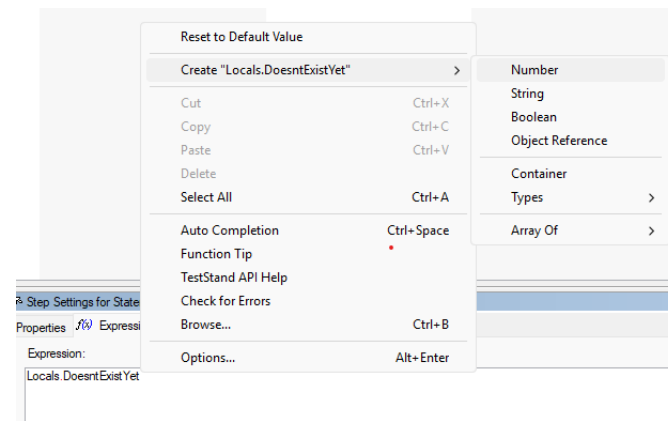
4.1.3 Working with Variables

4.1.3.1 Variables

To create a variable, right click under the desired scope in the Variables Pane. Then select the desired type from the resulting dropdown.

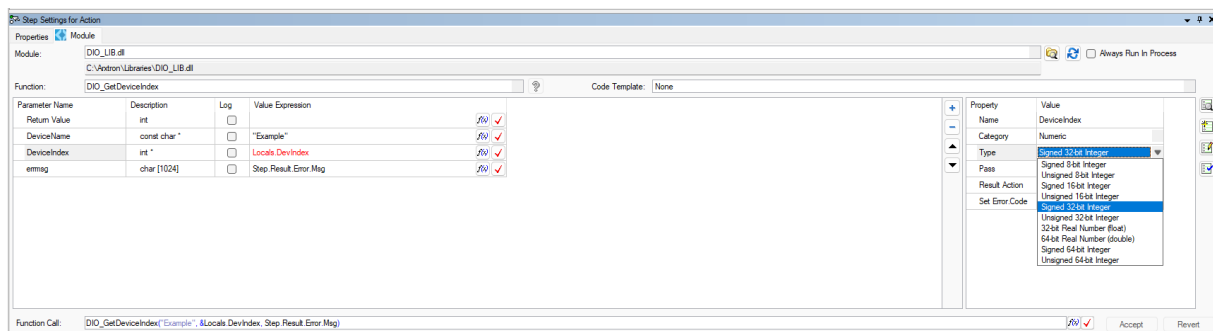


There is another useful way to create variables: type the desired variable name into any expression box, right click the variable name, and select **Create "variable name" > Desired variable type**.



4.1.3.2 Variables into Code Modules

It is very important that all variables being passed between TestStand and external C code have the appropriate type. To check how variables are being passed into a step, select the step in the Steps Pane, which will open the Step Settings Pane. Then, select a parameter to view its properties on the right side of the pane.



In this case, it would be important to ensure that `DIO_GetDeviceIndex()` from `DIO_LIB.dll` is supposed to take a pointer to a **signed 32-bit integer** for the parameter `DeviceIndex`.

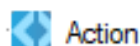
4.2 Test Steps

4.2.1 Test Step Types

TestStand contains many step types, which can be used to build test sequences. Here are a few of the most commonly used step types:

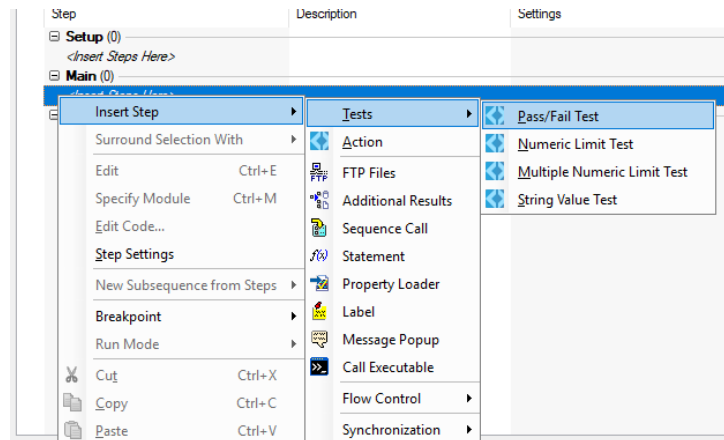
4.2.1.1 Action

Calls a code module that does not perform a test. For example, an action step might initialize an instrument to prepare it for testing.



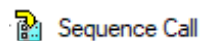
4.2.1.2 Pass/Fail

Calls a code module that makes its own pass/fail determination. Note that this module does not appear by default in the Insertion Palette; to insert a pass/fail step, right click the Steps Pane and select **Insert Step > Tests > Pass/Fail Test**. Within this menu are a few other types of code module tests.



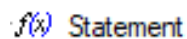
4.2.1.3 Sequence Call

Calls another sequence in the current sequence file or in another sequence file.



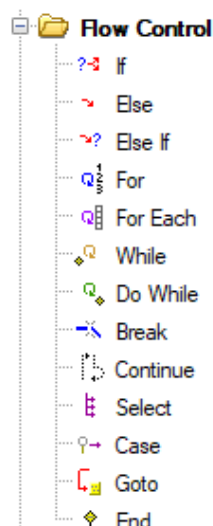
4.2.1.4 Statement

Executes a TestStand expression. For example, a statement step can be used to increment the value of a local variable. Note that statement steps do not return a pass/fail status by default; step status can be set in the Step Settings Pane under **Properties > Expressions > Status Expression**.



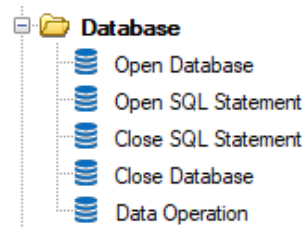
4.2.1.5 Flow Control

Flow control step types implement standard programming branching statements.



4.2.1.6 Database

Database step types can be used to easily communicate with databases, which is often useful for logging functionality.



4.2.2 Working with Test Steps

4.2.2.1 Creating a Test Step

To create a test step, simply drag and drop from the Insertion Palette into the Steps Pane. Alternatively, right click within the Steps Pane and select **Insert Step > Desired step type**.

4.2.2.2 Editing a Test Step

To edit a step, select it in the Steps Pane and navigate to the Step Settings Pane if it is not already open. Within the Step Settings Pane, there are typically (but not always) two tabs: **Properties** and **Step-Specific Settings**. Take some time to try experimenting with these tabs within multiple step types.

4.2.2.2.1 Step Properties

The **Properties** tab is a standard set of properties that appear in all TestStand step types.

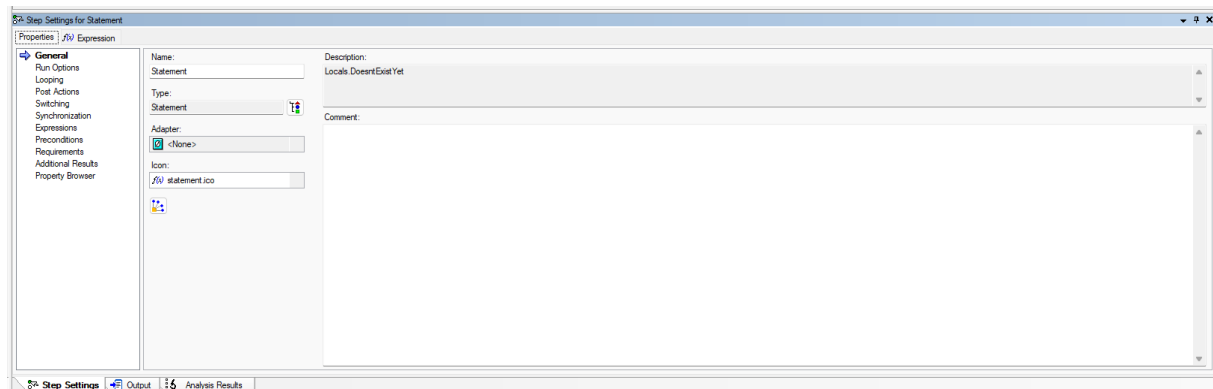


Figure 4.2 Step Properties Tab

There are a few key properties to note, all of which are used often at Arxtron (see [Test Step Flow](#) for more details):

- **Post Actions:** Determines how the sequence should behave depending on the result of the current test step.
- **Expressions:** Pre- and Post-Expressions are TestStand expressions that will run before and after the execution of the current step. The Status Expression determines the output status of the test (pass/fail)
- **Preconditions:** A TestStand expression that determines whether or not the current step is run in the sequence, via a logical condition.

4.2.2.2 Step-Specific Settings

Step-Specific Settings vary between test step types.

For example, an Action test step's settings primarily concerns the passing of parameters between TestStand and the code module.

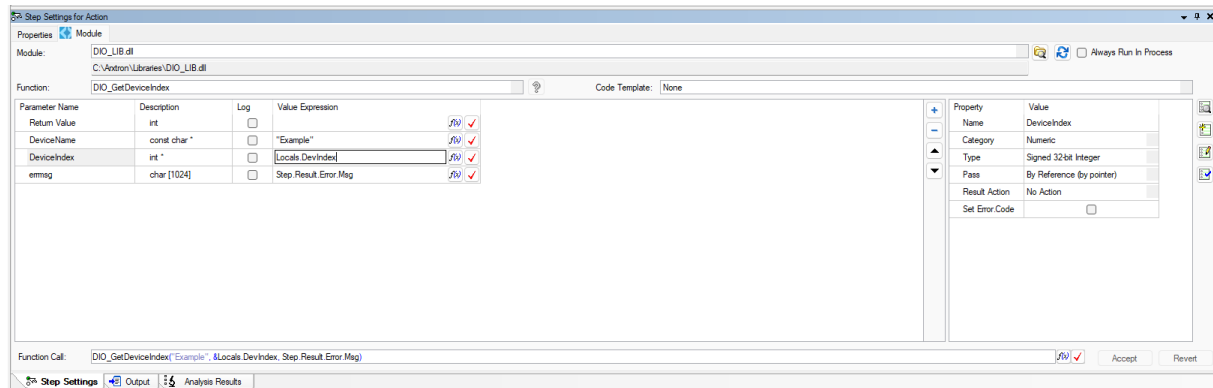


Figure 4.3 Step-Specific Settings Tab for an Action Test Step

4.3 Types

TestStand supports the creation of custom variable and step types. This functionality can be accessed from the Types Pane.

4.3.1 The Types Pane

The Types Pane can be opened via **View > Types** or by pressing **Ctrl+T**.

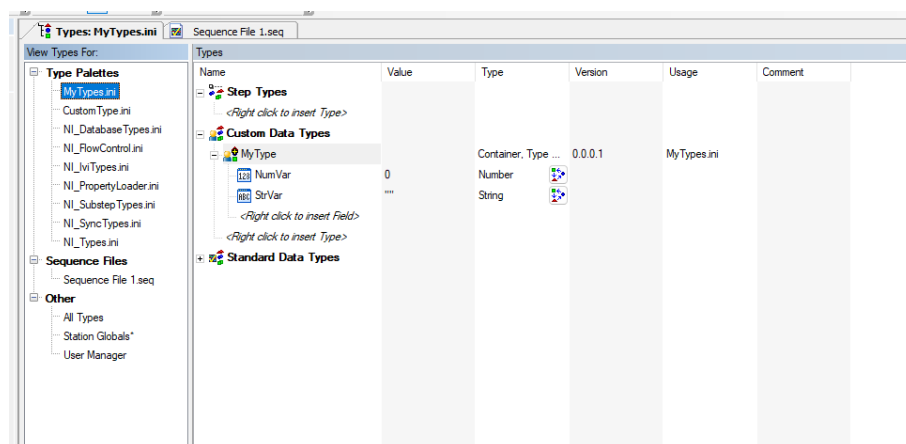


Figure 4.4 Types Pane

4.3.2 Working with Types

4.3.2.1 Creating a Type

To create a type, simply right click under the desired type list and follow the resulting pop-up menu.

4.3.2.2 Data Type Properties

When creating a custom data type, it is important that you correctly configure its properties. To do so, right click the data type and select **Properties**.

The most important property to be aware of is C Struct Passing, as it allows variables of this data type to be passed into C code modules as C structs. To set up this functionality, follow these steps:

1. Click the checkbox titled "Allow Objects of this Type to be Passed as Structs"
2. For each field in the data type, ensure that the type matches the type defined by the corresponding code module parameter.

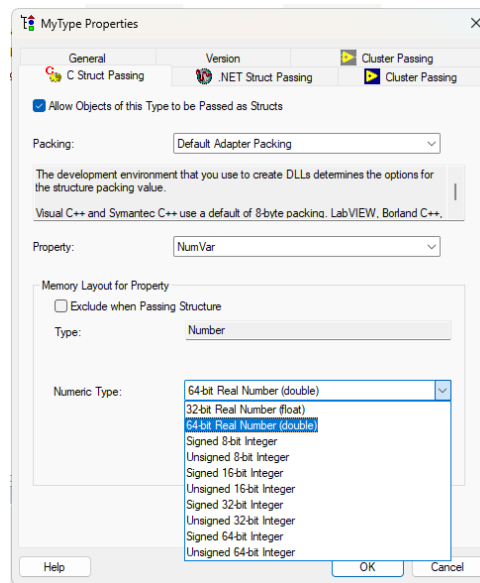


Figure 4.5 Data Type Properties Window

4.4 TestStand Flows

4.4.1 Sequence Flow

TestStand follows a standard order of sequences within a sequence file, outlined by the following flowchart. If the following sequences do not appear in TestStand, navigate to **Edit > Sequence File Callbacks**. From that menu, you can add default TestStand callback sequences.

1. **ProcessSetup**: Used for program setup (e.g. starting threads, initializing libraries).
2. **PreBatch**: If batch model is active, runs before each batch.
3. **PreUUT**: Runs before each UUT.
4. **MainSequence**: Primary test sequence to be run.
5. **PostUUT**: Runs after each UUT.
6. **PostBatch**: If batch model is active, runs after each batch.
7. **ProcessCleanup**: Used for program cleanup.

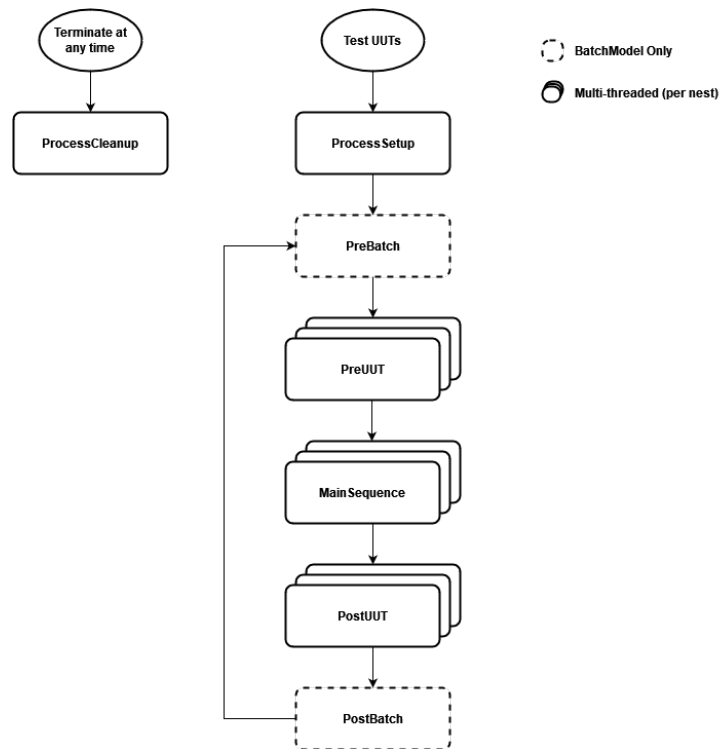


Figure 4.6 TestStand Sequence Flow

4.4.2 Test Step Flow

TestStand also follows a standard procedure within each test step, as outlined below.

1. **Preconditions:** See [Step Properties](#).
2. **Loop Initialization Expression:** Used to initialize loop variables (e.g. `int i = 0`).
3. **Pre-Expression:** See [Step Properties](#).
4. **Module Action:** Main module action.
5. **Post-Expression:** See [Step Properties](#).
6. **Status Expression:** Determines pass/fail status of step.
7. **Loop Increment Expression:** Increments loop variables (e.g. `i++`).
8. **Loop While Expression:** Checks loop condition (e.g. `i < 10`).
9. **Loop Status Expression:** Determines pass/fail status of loop.
10. **Post Actions:** See [Step Properties](#).

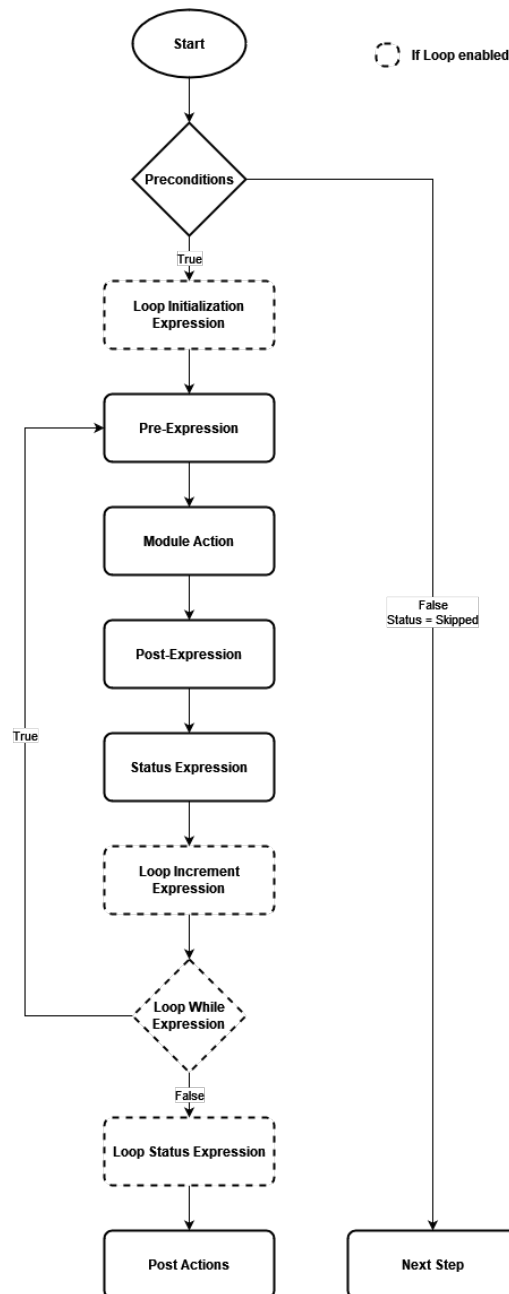


Figure 4.7 TestStand Test Step Flow

Chapter 5

Running a Test Sequence

5.1 Program Execution

5.1.1 Step Run Modes

Right-clicking on a test step and navigating to **Run Mode** reveals the available step run modes. These step modes often prove useful while debugging.

- **Force to Pass:** the test step always passes regardless of the actual test result.
- **Force to Fail:** the test step always fails regardless of the actual test result.
- **Skip:** the test step is skipped during sequence execution (used to "comment out" steps).
- **Normal:** the test step runs normally.

5.1.2 Running the Sequence

There are a few ways to run a TestStand sequence. To see these options, hover over **Execute**.

- **Test UUTs:** Runs the sequence utilizing the station model.
- **Single Pass:** The same as **Test UUTs**, but only tests one UUT.
- **Run MainSequence:** Runs the MainSequence directly without using the station model (useful for debugging).

5.1.3 Running Selected Steps

To run a subset of steps from a sequence, select the desired steps within the Steps Pane and right click, selecting **Run Selected Steps**. This can also be done from **Execute > Run Selected Steps**.

5.2 Debugging

Debugging a TestStand sequence can be approached similar to conventional code debugging.

5.2.1 Sequence Analyzer

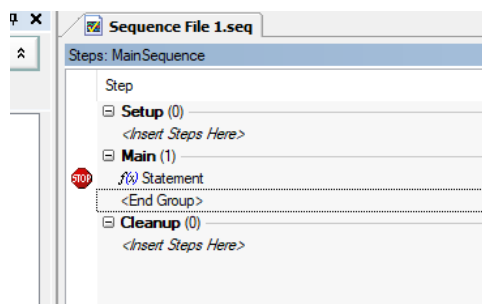
TestStand's sequence analyzer is a tool that will attempt to point out potential issues in a sequence. It can be run via **Debug > Sequence Analyzer > Analyze "SequenceFileName.seq"**. Additionally, it can be automatically set to run before any execution, via **Debug > Sequence Analyzer > Toggle Analyze File Before Executing**.

Sequence analysis results will populate in the Analysis Results Pane, which is in the same location as the Step Settings Pane.

5.2.2 Breakpoints & Watch Expressions

Breakpoints can be set by navigating to **Debug > Breakpoints/Watches**, and adding a breakpoint via the resulting menu. Note that watch expressions can also be set from within the menu.

Alternatively, clicking within the white space directly to the left of a test step will enable a breakpoint.



5.2.3 Step Commands

While executing a sequence with breakpoints, standard debugging step commands can be used. They can be found under **Debug**; here are some of the most useful ones:

- **Resume**: Resumes execution until the next breakpoint is reached (F5).
- **Step Over**: Steps over the current test step without resuming execution (F8).
- **Step Into**: Steps into the current test step (F10). Especially useful for stepping into the source code of a code module.

Chapter 6

Tips & Tricks

The following tips and tricks are helpful to know.

- **Help Menu:** TestStand (weirdly) has two help menus, which can be accessed via **F1** and **Ctrl+F1**. The former provides context-sensitive help, while the latter contains all help documentation. It is recommended that you use the full help menu (**Ctrl+F1**).
- **Running Selected Test Steps:** A set of specific test steps can be independently run, by selecting them in the Step Pane and right clicking, then selecting **Run Selected Steps** from the dropdown.
- **Unload All Modules:** All currently held modules (typically .dlls) can be released by selecting **File > Unload All Modules**.
- **Reset UI Configuration:** TestStand's UI can be reset to its default state by selecting **View > Reset UI Configuration**.
- **Lock UI Configuration:** TestStand's UI can be locked to prevent further changes. To toggle this setting, select **View > Lock/Unlock UI Configuration**.
- **Drag & Drop Variables:** TestStand variables can easily be dragged and dropped into most menus and text boxes. For example, instead of typing "Locals.ExampleLongVariableName" into a statement test step's expression editor, you can simply drag the variable from the Variables Pane.
- **Expression Browser:** The expression browser contains a variety of tools that can be helpful if you get stuck while writing an expression. The red checkmark button is also useful, as it checks the current expression for errors.

Chapter 7

TestStand API

WIP **NI** Documentation

Chapter 8

Arxtron TestExec

WIP

Chapter 9

Change Log

Version	Author	Date	Changes
1.↔ 0.0	Michael Harhay	24/3/2026	Initial Release

Index

Arxtron TestExec, [25](#)

Change Log, [27](#)

Configuration & Options, [5](#)

Developing a Test Sequence, [9](#)

Running a Test Sequence, [19](#)

Start Here, [1](#)

Terminology, [3](#)

Test Steps, [11](#)

TestStand API, [23](#)

TestStand Flows, [15](#)

Tips & Tricks, [21](#)

Types, [14](#)

Variables, [9](#)